

A genetic algorithm for total graph coloring

Arindam Dey^a, Aayush Agarwal^b, Pranav Dixit^c, Pranav Dixit^d, Hoang Viet Long^{e,f,*}, Frank Werner^g, Tandra Pal^h and Le Hoang Sonⁱ

^a*Department of Computer Science and Engineering, Saroj Mohan Institute of Technology, Hooghly, India*

^b*Department of Computer Science and Engineering, National Institute of Technology, Durgapur, India*

^c*Department of Computer Science and Engineering, National Institute of Technology, Durgapur, India*

^d*Department of Computer Science and Engineering, National Institute of Technology, Durgapur, India*

^e*Division of Computational Mathematics and Engineering, Institute for Computational Science, Ton Duc Thang University, Ho Chi Minh City, Vietnam*

^f*Faculty of Mathematics and Statistics, Ton Duc Thang University, Ho Chi Minh City, Vietnam*

^g*Faculty of Mathematics, Otto-Von-Guericke University, Magdeburg, Germany*

^h*Department of Computer Science and Engineering, National Institute of Technology, Durgapur, India*

ⁱ*VNU Information Technology Institute, Vietnam National University, Hanoi, Vietnam*

Abstract. A genetic algorithm (GA) belongs to the class of evolutionary algorithms and it is one of the most studied heuristic algorithms to solve graph coloring problems. In this paper, we propose a new GA algorithm for the total graph coloring problem. To the best of our knowledge, no algorithm based on a GA exists in the literature for total graph coloring. In the proposed approach, a novel encoding scheme is introduced, where all the edges and vertices of the graph are represented in a chromosome without any repetition. For the initialization of the population, a greedy algorithm is used to determine the total number of colors required for a total coloring of the graph. The number of colors is used as the fitness value of a chromosome which depends on the sequence of vertices and edges representing the chromosome. We introduce a convergence criteria for GA based on the total coloring conjecture. A two-point crossover and mutation operations, suitable for total coloring, are suggested. The proposed algorithm is applied on some well-known and standard graphs. In our computational tests, graphs are used with a maximum number of 690 vertices and 6650 edges of the graph, respectively. The proposed algorithm determines an optimal solution for 21 graphs among the 27 graphs. The solution of remaining the 6 graphs is near optimal and differs by at most one unit from the optimal value. The results show the effectiveness of the proposed approach.

Keywords: Total coloring, total chromatic number, total coloring conjecture, genetic algorithm, crossover, mutation

1. Introduction

The total coloring problem (TCP) is one of the important problems in graph theory. A total coloring of a graph G is required to color all vertices and edges of G in such a way that no two adjacent ver-

tices and adjacent edges have the same color and the color of each vertex is distinct from those of its incident edges. The minimum number of colors required for a total coloring of a graph is known as the total chromatic number (TCN). It has applications in many real-world problems like scheduling [8], partitioning [8], computer networking [8], etc.

As an example, consider the problem of scheduling the final round for a boxing tournament. We assume

*Corresponding author. Hoang Viet Long. E-mail: hoangvietlong@tdtu.edu.vn.

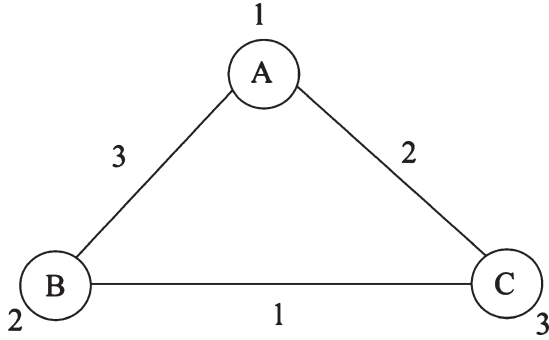


Fig. 1. A boat.

that 3 players are selected for the final round and each pair of the players have a match. For each of the players during the round, one day is assigned for his/her practice. Assume that there is only one practice ring. Thus, different days have to be assigned to different players for their practice and the day for practice of a player needs to be different from the day of his match. We need to find the minimum number of days required for the final round. This problem can be modeled as a total coloring problem. The graph, shown in Fig. 1, represents this problem, where each player is represented by a vertex, and two vertices are connected by an edge if the corresponding players have a match between them. The color, assigned to each vertex, represents the practice day of this player, and the edge color represents the day of the match between two players represented by the two end vertices of that edge. A total coloring ensures the following:

- (1) Each of the edges incident on a vertex have different colors as no player can play in two matches in the same day.
- (2) Two adjacent vertices should have different colors, because two players cannot practice in the same day.
- (3) No vertex and the edges incident on it are assigned to the same color because the day for the match and the day for practice have to be different.

The TCN depends on the order or sequence in which the vertices and edges are considered during coloring the graph. All possible sequences of vertices and edges and its corresponding number of colors, required for a total coloring the graph, constitute the search space. The TCP is to find out the sequence for which the required number of colors is minimal. If the numbers of vertices and edges are very large in a graph, the corresponding search space becomes also

very large. In such cases, computing exactly the TCN of a graph is a hard problem because it would take too much time. The TCP is an NP-complete problem, and heuristic algorithms are needed to find the TCN approximately. However, to the best of our knowledge, there exist no metaheuristic algorithm for this problem. Subsequently, we propose a Genetic Algorithm (GA) to find the TCN.

Genetic Algorithm (GA) is a probabilistic heuristic search based on the concepts of natural genetics. It is inspired by Darwin's law of survival of the fittest and mimics the process of natural evolution. A GA works with a population of possible solutions. A chromosome (or individual) represents a possible solution. A GA recombines the structural information to locate new points in the search space with the hope to improve the solutions. Two basic genetic operators, crossover and mutation, are used for a recombination. In the selection process, less fit solutions are replaced by fitter solutions. Thus, a GA gradually evolves towards an optimal solution. The fitness value of a chromosome in a GA is the value of the objective function. As the termination criterion of the algorithm, one may use e.g. either a specified maximum number of generations, a satisfactory fitness value of the best solution, or an assigned time limit.

In this study, we propose an algorithm for the TCP based on GA. We introduce an encoding method to represent all vertices and edges of an input graph in a chromosome. The initial population is generated by means of a greedy algorithm, which also computes the TCN of each chromosome. The TCN is used as the fitness of a chromosome. A lower bound of the TCN of a graph is $\Delta(G) + 1$, where $\Delta(G)$ represents the maximum degree among all the nodes of the graph G , discussed below in Section 2. It is used as one of the termination criteria to make the algorithm faster. We use a two-point crossover and a mutation applied successfully also to other permutation problems [21].

This paper is organized as follows. Section 2 briefly reviews some basics related to this study. In Section 3, we describe the proposed algorithm for the TCP based on a GA. Details of the implementation of the algorithm and the results of the proposed algorithm are presented in Section 4. Finally, we conclude the paper in Section 5.

2. Preliminaries

Let $G = (V, E)$ be a graph, where V represents the set of vertices and E represents a non-empty set of

edges. Two vertices u and v , $u, v \in V$ are called adjacent if $\{u, v\}$ is an edge of G . Two edges are incident to each other if both have a common end vertex. The degree of a vertex v is the total number of edges incident on it, except a loop. The maximum degree of G is denoted as $\Delta(G)$, which is the maximum degree of a vertex among all the vertices. A vertex coloring of a graph G requires to assign colors to all vertices of G such that different colors are assigned to every pair of adjacent vertices. Similarly, an edge coloring of a graph G requires to assign colors to the edges of the graph G such that different colors are assigned to two incident edges. The third type is a total coloring of a graph G , introduced by M. Behzad [13]. A total coloring [27] of $G = (V, E)$ requires to assign colors to all vertices and edges of the graph G such that different colors are assigned to two adjacent vertices and also to two incident edges, and the color of any vertex $v \in V$ is different from those of all edges $e \in E$, which have v as a common vertex. The minimum number of colors required for a total coloring of a graph is known as the total chromatic number (TCN) which is denoted by $\chi^T(G)$.

In this section, we present some basics of the TCP, associated with the proposed algorithm. In our study, we use the symbols, $\Delta(G)$ for the maximum degree, $\chi^T(G)$ for the TCN, n for the number of vertices, P_n for the path graph, C_n the cycle graph and K_n the complete graph, each with n vertices.

Behzad [13] and Vizing [20] independently proposed the following conjecture for a graph G

$$\chi^T(G) \leq \Delta(G) + 2 \quad (2.1)$$

This conjecture is known as the total coloring conjecture (TCC). $\chi^T(G)$ cannot be less than $\Delta(G) + 1$ since the number of colors associated with a vertex having the maximum degree and the edges incident on it is $\Delta(G) + 1$. So, a lower bound on the value $\chi^T(G)$ is $\Delta(G) + 1$.

Theorem 2.1. [13, 20].

The Total Coloring Conjecture is

$$\Delta(G) + 1 \leq \chi^T(G) \leq \Delta(G) + 2.$$

Many researchers have paid a lot of attention to prove the TCC for graphs of different sizes and different types. M. Resenfeld [14] verified the TCC for bipartite graph, complete tripartite graph and complete balanced k - partite graph. Kostochka [1–3] verified the TCC for graphs for which $\Delta(G) = 4$ or 5. The number $\chi^T(G)$ of a planar graph G is equal to its lower bound, i.e., $\chi^T(G) = \Delta(G) + 1$.

The TCC for a planar graph with $\Delta(G) \geq 9$ has been verified by Borodin [15]. Shen *et al.* [11] proved that $\chi^T(G)$ of a planar graph with $\Delta(G) = 8$ is 9 if G has no intersecting triangle. According to Hou *et al.* [9], a graph G with $\Delta(G) = 8$ is planar and if G has no 5-cycle or no 6-cycle, then $\chi^T(G) = 9$. The TCC has also been verified for the Halin graph [12], an interval graph [22] and a series-parallel graph [19].

Theorem 2.2. [23].

The TCN of the path graph P_n is

$$\chi^T(P_n) = \Delta(P_n) + 1 = 2 + 1 = 3.$$

Theorem 2.3. [23].

The TCN of the cycle graph C_n is

$$\chi^T(C_n) = \begin{cases} 3 & \text{if } n \bmod 3 = 0 \\ 4 & \text{otherwise.} \end{cases} \quad (2.2)$$

Theorem 2.4. [8].

The TCN of the complete graph K_n is

$$\chi^T(K_n) = \begin{cases} \Delta(K_n) + 2 & n = 2k, n = 1, 2, 3, \dots, n \\ \Delta(K_n) + 1 & n = 2k + 1, n = 1, 2, 3, \dots, n \end{cases} \quad (2.3)$$

A genetic algorithm has been used for the vertex coloring problem first by Davis *et al.* [10]. They suggested to explore the order of vertices first and then to assign the colors to the vertices based on the order using a greedy method. This algorithm has later been modified and improved by various local search algorithms [6, 7, 16–18]. However, there exists no algorithm for the edge coloring problem and TCP based on a GA in the literature to the best of our knowledge.

3. Proposed Algorithm for TCP

Given an input graph, the proposed GA finds the TCN for the graph and the corresponding coloring. Since the proposed GA works with permutations of vertices and edges, we apply components of GAs which have successfully been applied to several other permutation problems, e.g. scheduling problems [21]. Next, we describe the chromosome encoding scheme, the fitness function, the selection, and the crossover and mutation operators used in our algorithm in more detail.

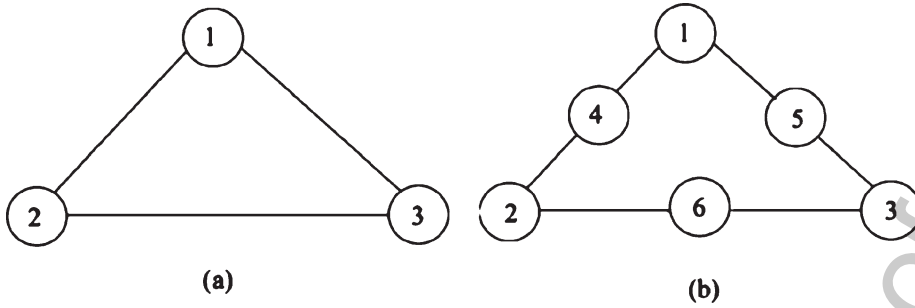


Fig. 2. (a) Original graph (b) Modified graph for the purpose of encoding.

3.1. Chromosome Encoding

A chromosome (or individual) represents a possible solution. For the encoding purpose, the input graph is modified by adding one more vertex between each pair of two adjacent vertices of the original graph. So, the number of vertices in the modified graph is the sum of the number of edges and the number of vertices in the original input graph. For example, a graph with 3 edges and 3 vertices is shown in Fig. 2. Let n be the number of vertices in the modified graph. All the vertices of the modified graph are numbered starting from 1 to n . A chromosome is represented by an ordered sequence (permutation) of the n vertices of the modified graph. Thus, the chromosome length is n , i.e., the total number of vertices and edges of the original graph. Moreover, the population size $POPSIZE$ fixes the number of chromosomes contained in the initial population.

3.2. Fitness Function

The fitness of any chromosome describes the value of objective of a specific problem. For finding fitness value, the chromosome needs to be first decoded and the objective function of a problem has to be measured. The fitness score not only describes how efficient the solution is, but also corresponds to how close the chromosome is to the optimal one. The fitness of a chromosome is defined by the number of different colors required to color all vertices of the modified graph such that all adjacent vertices and adjacent to adjacent (i.e., a path contains exactly two edges and a vertex between them) vertices of the modified graph have different colors. The fitness value of a chromosome, i.e., the number of different colors, depends on the sequence of the vertices in this chromosome. We use a greedy

algorithm to find the fitness value of a chromosome. Given the adjacency matrix of a graph and an order of vertices $optorder$ in a chromosome, the greedy algorithm used to construct a coloring $C: \rightarrow \{1, 2, \dots, c\}$ of the chromosome is described below.

- Step 1. The color of a vertex of the chromosome depends on its position k in the chromosome. It starts from the first position. So, initially we have $k = 1$ and $colornumber = 1$, where $colornumber$ denotes the number of different colors, currently being used.
- Step 2. Consider the vertex of the chromosome at position k and denote this vertex as i . Vertex i is the current vertex to be colored.
- Step 3. Check whether the vertex i is already colored or not. If the vertex i is not colored, then find the minimum value of j between 1 and n such that j is not present in the list of adjacent vertices or adjacent to adjacent vertices of vertex i . The color corresponding to the value of j is used to color the vertex i . If j is greater than $colornumber$, then assign the value of j to $colornumber$.
- Step 4. The list of colors of adjacent vertices is updated for all vertices of the graph.
- Step 5. Increment k by 1.
- Step 6. If k is not equal to n , then perform Step 2 to Step 5. Otherwise, return $colornumber$, which is used as the fitness value of the chromosome.

A list of colors of the adjacent vertices is associated with all vertices of the graph. Initially, the lists corresponding to all vertices in a chromosome do not contain any color. Another list contains the set of different colors following a particular order from a lower one to a higher one. In this method, we start to assign

a color to the vertices in the order as they occur in the chromosome. If the color to be assigned to a vertex is the same as that of its adjacent vertices or its adjacent to adjacent vertices, then we assign a different color of higher order, not yet used, to this vertex. The number of colors used for coloring a chromosome represents the fitness of this chromosome. We keep track of the minimal number colors already used for coloring the chromosomes.

3.3. Selection

In this study, we use a Roulette wheel selection, where the more fitted chromosomes have higher probabilities to be selected compared to less fitted chromosomes to create the mating pool.

3.4. Crossover

In the crossover, the chromosomes in the mating pool undergo a recombination process to create new chromosomes for the next generation. Two parent chromosomes, selected randomly from the mating pool, generate two chromosomes as their children by crossover. A simple two-point crossover is not suitable for the TCP because it can make the children with repeated vertices causing some other vertex absent in it. It turns out that a modification often used for permutation problems works successful. This modified two-point crossover method [5, 21] is known as LOX crossover. It is easier as well as faster than PMX.

Let the two parent chromosomes *Parent1* and *Parent2* are selected from the mating pool, and they generate two children, *Child1* and *Child2*, by a LOX crossover as follows (Fig. 3 shows an example of this crossover for $n = 6$).

- Step 1. Two random positions *point1* and *point2* are chosen from the chromosome as cut points. Let $point1 < point2$.
- Step 2. The vertices from *point1* to *point2* of *Parent2* are copied into the same position of *Child1*.
- Step 3. The remaining vertices of *Parent2* are rearranged in the available positions of *Child1* following their order in *Parent1*.
- Step 4. The vertices from *point1* to *point2* are copied from *Parent1* into *Child2*.
- Step 5. The remaining vertices of *Parent1* are rearranged in the available positions of *Child2* following their order in *Parent2*.

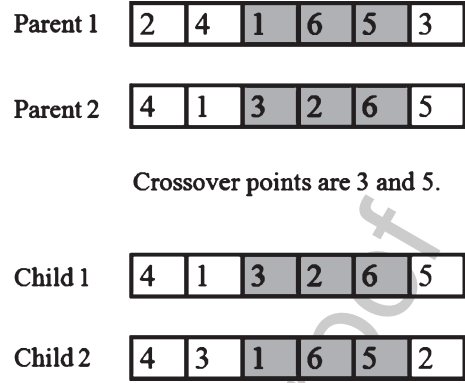


Fig. 3. Illustration of the LOX crossover.

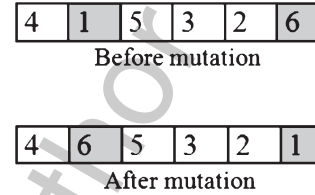


Fig. 4. Illustration of a mutation, where the randomly selected points are 2 and 6.

3.5. Mutation

After a possible crossover, a mutation can be applied to a chromosome. A mutation introduces diversity in the GA to avoid local convergence. The mutation operator used in the proposed GA is described in the following two steps (Fig. 4 shows an example of the mutation operator for $n = 6$).

- Step 1. Choose two random points say p_1, p_2 from a randomly selected chromosome of the current population.
- Step 2. Exchange the vertices of position p_1 and position p_2 .

4. Implementation and Results

First, the proposed algorithm for a total coloring has been tested on three different types of small graphs: path graphs, complete graphs and circuit graphs with different numbers of vertices and edges. For these computational tests, the number of generations, population size, crossover probability and mutation probability are set to 10, 5, 0.8 and 0.1 respectively. For each of these graphs, our algorithm

Table 1
Results of the proposed GA for some known graphs

Graph Name	n_v	n_e	TCN	c
Path Graph	6	5	3	3
Path Graph	12	11	3	3
Path Graph	20	19	3	3
Complete Graph	6	7	7	7
Complete Graph	7	7	7	7
Circuit Graph	12	4	3	3
Circuit Graph	10	5	4	4

produces an optimal solution, i.e., the minimum number of colors [14, 24] for a total coloring. Table 1 shows the summary of our experiments. Here and in the following tables, n_v represents the number of vertices, n_e represents the number of edges and c denotes the minimum number of different colors obtained by the algorithm.

Next, the proposed genetic algorithm has been tested on 27 popular standard graphs [4]. For each of these graphs, the proposed GA has been executed 20 times with three different pairs ($NGEN$, $POPSIZE$) of number of generations and population size: (30, 5), (30, 20), (100, 20). We have chosen a crossover probability of 0.8 and a mutation probability of 0.1. The tested graphs and the corresponding results are shown in Table 2. For 21 graphs among these 27 graphs, the GA gives definitively an optimal solution and for the remaining 6 graphs (row numbers 1, 2, 11, 14, 15 and 17), the results follow the total coloring conjecture (TCC), mentioned in (1). We cannot claim that the solutions for those 6 graphs as optimal because their TCN values are not available in the literature. However, our results show that either they are also optimal or differ only by one from the optimal value. We did not find any algorithm which shows better results.

In these experiments, for 22 graphs among the 27 graphs the same values of TCN for all of three different sets of generations and population sizes. However, for 5 graphs we obtained different results for different sets of parameter values. Those graphs are shown in bold face in Table 2. For those 5 graphs, we present the minimum value in Table 2 among all the results. In Table 3, we show the percentage of the number of executions to get the minimum number of colors for those 3 sets of pairs of number of generations and population size. Case 1, Case 2 and Case 3 are used to represent the three different sets of pairs (30, 5), (30, 20) and (100, 20), respectively. It shows that, even if we do not get the optimal solution, proposed GA produces near optimal solution, which is the mini-

Table 2
Results of the proposed GA for 27 standard graphs

Sl. No.	Graph Name	n_v	n_e	$\Delta(G)$	c
1	<i>Tetrahedron</i>	4	6	3	5
2	<i>The Kuratowski Bipartite Graph</i>	6	9	3	5
3	The Octahedron	6	12	4	5
4	The Bondy-Murty Graph	7	12	4	5
5	The Wheel Graph	8	12	7	8
6	The cube	8	14	3	4
7	The Petersen Graph	10	12	3	4
8	The Bondy-Murty graph	11	15	8	9
9	The Grötsch graph	11	28	5	6
10	The icosahedron	12	30	5	6
11	<i>The Chvatal graph</i>	12	20	4	6
12	The bondy-Murty Graph G-3	14	21	3	4
13	The bondy-Murty Graph G-4	16	15	3	4
14	<i>The Ramsey Graph</i>	17	68	8	10
15	<i>The dodecahedron</i>	20	30	3	5
16	The Mycielski 5-Chromatic Graph	23	71	11	12
17	<i>The Grünbaum Graph</i>	25	50	4	6
18	The map of India	30	57	9	10
19	The Mycielski 6-Chromatic Graph	47	236	23	24
20	2-Fullins-3	52	201	15	16
21	3-Fullins-3	80	346	12	13
22	1-Fullins-4	93	593	32	33
23	The Mycielski 7-Chromatic Graph	95	755	47	48
24	4-Fullins-3	114	541	23	24
25	1-Fullins-5	282	3247	95	96
26	3-Fullins-4	405	3524	84	85
27	4-Fullins-4	690	6650	119	120

Table 3
Percentage how often the minimum number of colors has been obtained by the proposed GA

S.L	Graph Name	Case 1	Case 2	Case 3
1	The Octahedron	35	50	95
2	The cube	25	50	75
3	The Petersen Graph	25	60	75
4	The icosahedron	0	5	85
5	The bondy-Murty Graph G-3	0	15	75

num number of colors+1 in every execution. Table 3 also shows that, if the number of generations and population sizes increase, the number of times to get the minimum color of a graph also increases. It turns out that the parameter settings according to case 3 are sufficient to ensure a high quality of the final solution. The termination condition, mentioned in Section 3.6, makes the algorithm faster and avoids unnecessary generations. It also reduces the problem for large population and number of generations. Since we did not find any algorithm based on a GA for the total coloring problem, we could not provide any comparison of our results with any other similar algorithm from the literature.

4.1. Termination condition

The proposed GA terminates when the specified maximum number $NGEN$ of generations is reached. For the TCP, the value of TCN is either $\Delta(G) + 1$ or $\Delta(G) + 2$. Here, we use a convergence criterion based on the value of TCN: If some chromosome has the fitness value $\Delta(G) + 1$, then it is an optimal solution. Thus, if some chromosome attains the fitness value $\Delta(G) + 1$ before the GA reaches the maximum number $NGEN$ of generations as specified, the execution of the GA is terminated.

In this study, we develop a genetic algorithm for the total coloring problem. However, in practice, uncertainty cannot be avoided. In future, we will try to introduce a genetic algorithm for total coloring problem in uncertain environment. Here, we have used Roulette wheel selection method in our proposed algorithm. However, Roulette wheel [25] has problems if the set of genomes has a DC offset. For example 3 chromosomes with fitness values of 7, 8 and 9 would have almost exactly the same probability of being chosen, but subtract 6 of each and we get probabilities of 1/4, 1/3 and 1/2. The probabilities become closer together as the DC offset increases. Tournament does not have this property. In future, we will try to use tournament selection method in our proposed genetic algorithm. Uniform Crossover is shown by Syswerda [26] to be optimal and is anyway generally better than PMX and LOX crossover. We also try to use the Uniform Crossover in our proposed algorithm. Despite the need for future work, the proposed algorithm described in this work is important initial contributions to domain of genetic algorithm and total graph coloring problem.

5. Conclusions

The main contribution of this study is to develop a genetic algorithm for the total coloring problem. Though the total coloring problem is mostly studied as a theoretical problem in graph theory, many real-world problems may be modeled by the total coloring problem. An example has been described in Section 1. For most of the graphs, considered in this study, our algorithm provides an optimal solution. For the remaining few graphs, the results are very close to an optimal solution, which are obtained within a reasonable time even for graphs with a large number of vertices and edges. The greedy algorithm, used to color the chromosomes in the initial population,

is also time consuming. In the future, we intend to consider these limitations as well as the application of the algorithm to real-world problems in an uncertain environment, where the problem will be modeled by a fuzzy graph using advanced fuzzy algorithms [28–30].

References

- [1] A. Kostochka, The total coloring of a multigraph with maximal degree 4, *Discrete Mathematics* **17**(2) (1977), 161–163.
- [2] A. Kostochka, Upper bounds of chromatic functions of graphs, *Doct Thesis* (1978).
- [3] A. Kostochka, The total chromatic number of any multigraph with maximum degree five is at most seven, *Discrete Mathematics* **162**(1) (1996), 199–214.
- [4] A. Dharwadker, The vertex coloring algorithm, available on http://www.geocities.com/dharwadker/vertex_coloring, (2006).
- [5] A. Dey, R. Pradhan, A. Pal and T. Pal, The fuzzy robust graph coloring problem, *Proc. of the 3rd International Conference on Frontiers of Intelligent Computing: Theory and Applications (FICTA)* **2014** (2015), 805–813.
- [6] C. Fleurent and J.A. Ferland, Genetic and hybrid algorithms for graph coloring, *Annals of Operations Research* **63**(3) (1996), 437–461.
- [7] D. Costa, A. Hertz and C. Dubuis, Embedding a sequential procedure within an evolutionary algorithm for coloring problems in graphs, *Journal of Heuristics* **1**(1) (1995), 105–128.
- [8] H.P. Yap, Total colourings of graphs, *Lecture Notes in Mathematics* **1623** (1984).
- [9] J. Hou, Y. Zhu, G. Liu, J. Wu and M. Lan, Total colorings of planar graphs without small cycles, *Graphs and Combinatorics* **24**(2) (2008), 91–100.
- [10] L. Davis, et al., Handbook of genetic algorithms **115** (1991).
- [11] L. Shen, Y. Wang, W. Wang and K.W. Lih, On the 9-totalcolorability of planar graphs with maximum degree 8 and without intersecting triangles, *Applied Mathematics Letters* **22**(9) (2009), 1369–1373.
- [12] L. Linzhong, X. Jiguo and Z. Zhongfu, On the vertex strong total coloring of graphs, *Mathematics in Economics* **15**(3) (1998), 52–55.
- [13] M. Behzad, Graphs and their chromatic numbers, Ph.D. thesis (1965).
- [14] M. Rosenfeld, On the total coloring of certain graphs, *Israel Journal of Mathematics* **9**(3) (1971), 396–402.
- [15] O.V. Borodin, On the total coloring of planar graphs, *J reine angew Math* **394** (1989), 180–185.
- [16] P. Calegari, G. Coray, A. Hertz, D. Kobler and P. Kuonen, A taxonomy of evolutionary algorithms in combinatorial optimization, *Journal of Heuristics* **5**(2) (1999), 145–158.
- [17] P. Galinier and J.K. Hao, Hybrid evolutionary algorithms for graph coloring, *Journal of Combinatorial Optimization* **3**(4) (1999), 379–397.
- [18] R. Dorne and J.K. Hao, A new genetic local search algorithm for graph coloring, *Proc. Parallel Problem Solving from NaturePPSN V* (1998), 745–754.
- [19] S.D. Wang and S.C. Pang, The determination of the total chromatic number of series-parallel graphs with (g) 4, *Graphs and Combinatorics* **21**(4) (2005), 531–540.

- [20] V.G. Vizing, Some unsolved problems in graph theory, *Russian Mathematical Surveys* **23**(6) (1968), 125–141.
- [21] F. Werner, A Survey of Genetic Algorithms for Shop Scheduling Problems, in: P. Siarry (ed.) *Heuristics: Theory and Applications*, Nova Science Publishers (2013), 161–222.
- [22] V. Bojarshinov, Edge and total coloring of interval graphs, *Discrete Applied Mathematics* **114**(1) (2001), 23–28.
- [23] Z. Zhang, X. Chen, J. Li, B. Yao, X. Lu and J. Wang, On adjacent-vertex-distinguishing total coloring of graphs, *Science in China Series A: Mathematics* **48**(3) (2005), 289–299.
- [24] Z. Zhang, H. Cheng, B. Yao, J. Li, X. Chen and B. Xu, On the adjacent-vertex-strongly-distinguishing total coloring of graphs, *Science in China Series A: Mathematics* **51**(3) (2008), 427–436.
- [25] G. Syswerda, Uniform Crossover in Genetic Algorithms, *Proceedings of Third International Conference on Genetic Algorithms* (1989), 2–9.
- [26] S. Jones and J. Hinde, Uniform Random Crossover, *Proceedings of the 2007 workshop on Computational Intelligence* (2007).
- [27] H. Wang, B. Liu, Y. Gu, X. Zhang, W. Wu and H. Gao, Total coloring of planar graphs without adjacent short cycles, *Journal of Combinatorial Optimization* **33**(1) (2017), 265–274.
- [28] L.H. Son and H. Fujita, 'Neural-Fuzzy with Representative Sets for Prediction of Student Performance, *Applied Intelligence* **49**(1) (2019), 172–187.
- [29] H.V. Long, M. Ali, L.H. Son, M. Khan and D.N. Tu, A novel approach for Fuzzy Clustering based on Neutrosophic Association Matrix, *Computers and Industrial Engineering*.
- [30] M. Ali, L.H. Son, M. Khan and N.T. Tung, Segmentation of Dental X-ray Images in Medical Imaging using Neutrosophic Orthogonal Matrices, *Expert Systems With Applications* **91** (2018), 434–441.

528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544